ELI — Capabilities And Actions

ELI v2.5.21

October 2026

Contents

ELI — Capabilities & Actions (with activation phrases) 1

Conversation & persona . . . 1
App & window control . . . 2
Input control . . . 4
Media . . . 4
Vision & screen . . . 4
Gaze (eye-tracking) . . . 5
Voice . . . 5
Files & notes . . . 6
Generation (docs/code) . . . 7
System status & info . . . 8
Grounded introspection . . . 8
Memory & profile . . . 10
Self-maintenance . . . 10
Plugins . . . 11
Tasks, time & planning . . . 12
Proactive . . . 13
Shell & advanced . . . 13
Aliases & internal actions (not directly user-triggered) . . . 13
Coverage . . . 14

ELI — Capabilities & Actions (with activation phrases)

Auto-generated 2026-10-20 from `capability_manifest.json` (231 capabilities; 214 routable or executor-backed). Regenerated by `eli/tools/registry/capabilities_doc.py` alongside the manifest.

How to read this: every row is a real action ELI can route to and execute. Activation phrases are **representative** — the router is paraphrase- and typo-tolerant; a phrase shows the *shape* that fires the action, not the only accepted wording. `Plugin` = backed by a plugin (rest are core executor+router). Voice users prefix with the wake word ("computer, …"). Generation actions (documents/scripts/projects) run the evidence-planner first (real code/web/memory analysis) before synthesising.

Conversation & persona

Action	What it does	Example activation phrase(s)	Source
CHAT	Open conversation / fallback; persona-bound reply	anything not matching a command · “how are you” · “tell me a story”	Core
EXPLAIN_LAST_RESPONSE	Explain why ELI gave its previous answer	“why did you say that” · “explain your last response”	Core
CLEAR_CHAT_HISTORY	Clear the current chat history	“clear chat history” · “start a new chat”	Core
PERSONA_LOCK_SET	Pin a persona/role for the session	“lock your persona to X”	Core
PERSONA_LOCK_CLEAR	Release a pinned persona	“clear persona lock”	Core
PERSONA_LOCK_STATUS	Show the current persona lock	“what persona are you locked to”	Core
PERSONA_REFRESH	Refresh/reload ELI’s persona from the latest profile	“refresh your persona” · “reload your persona”	Core
HELP	List what ELI can do	“help” · “what commands do you have”	Core
LIST_CAPABILITIES	Enumerate ELI’s capabilities	“what can you do” · “list your capabilities”	Core
SET_TONE	Shade ELI’s expressed tone/emotion (words+voice+face) without changing his core personality	“be comedic” · “talk street” · “sound more professional” · “use a deadpan tone”	Core
CLEAR_TONE	Drop a pinned tone — back to autonomous, situation-adaptive delivery	“go back to normal” · “be yourself” · “drop the tone”	Core
SET_COMMUNICATION_STYLE	Persist how you want ELI to speak to you (empty or a reset phrase clears it)	“speak concisely from now on”	Core
SET_AI_MODE	Set the reasoning mode (Quick, Normal, Advanced, Research, Expert); reached through the intent resolver or the mode selector	(no dedicated phrase)	Core
MULTI_COMMAND	Run several chained commands from one utterance, in order	“close steam and set an alarm for 7am” · “open spotify then play X”	Core

App & window control

Action	What it does	Example activation phrase(s)	Source
`OPEN_APP`	Launch an application (installs it if missing)	“open spotify” · “launch firefox”	Core
`CLOSE_APP`	Close an application	“close spotify” · “quit chrome”	Core
`FOCUS_APP`	Bring an app’s window to the front	“focus firefox” · “switch to the browser”	Core
`OPEN_BROWSER`	Open the default web browser	“open the browser”	Core
`OPEN_URL`	Open a URL in the browser	“open github.com” · “go to youtube.com”	Core
`OPEN_IDE`	Open the code IDE	“open the IDE”	Core
`OPEN_FILE_SYSTEM`	Open the file manager	“open files” · “open the file manager”	Core
`OPEN_SYSTEM_SETTINGS`	Open OS settings	“open system settings”	Core
`OPEN_AUDIO_SETTINGS`	Open sound settings	“open audio settings”	Core
`OPEN_POWER_SETTINGS`	Open power settings	“open power settings”	Core
`OPEN_NETWORK_BROWSER`	Open network settings	“open network settings”	Core
`OPEN_COMMUNICATION_HUB`	Open the comms hub (mail/chat apps)	“open communication hub”	Core
`OPEN_MEDIA_HUB`	Open the media hub	“open media hub”	Core
`TILE_WINDOWS`	Tile windows in a grid	“tile windows” · “2x2 grid” · “4x2”	Core
`MAXIMISE_WINDOW`	Maximise a named app’s window (args: name), or the focused one when none is named	“maximise firefox” · “maximise the window”	Core
`MINIMISE_ALL`	Minimise all / show desktop	“minimise everything” · “show desktop”	Core
`MINIMISE_WINDOW`	Minimise the focused window	“minimise the window” · “minimise this”	Core
`MINIMIZE_WINDOW`	Minimise the focused window (US spelling)	“minimize this window”	Core
`MINIMISE_APP`	Minimise a specific app’s window	“minimise spotify”	Core
`MINIMIZE_APP`	Minimise a specific app’s window (US spelling)	“minimize chrome”	Core
`HIDE_APP`	Hide an application’s window	“hide spotify” · “hide the window”	Core
`NEXT_WINDOW`	Switch to the next window	“next window”	Core
`PREVIOUS_WINDOW`	Switch to the previous window	“previous window”	Core
`RESTORE_WINDOWS`	Restore minimised windows	“restore windows”	Core
`SWITCH_WORKSPACE`	Switch virtual desktop/workspace	“switch to workspace 2”	Core
`SMART_HOME`	Control smart-home devices	“turn off the lights”	Core
`CONFIRM_PENDING_REMEDIATION`	Approve a pending fix/install offer	“yes” (to an “install X?” offer)	Core

Action	What it does	Example activation phrase(s)	Source
CANCEL_PENDING_REMEDIATION	Decline a pending fix/install offer	“no” (to an “install X?” offer)	Core

Input control

Action	What it does	Example activation phrase(s)	Source
VOLUME	Adjust/mute system volume	“volume up” · “mute” · “set volume to 30”	Core
KEYBOARD	Send keystrokes / type text	“press enter” · “type hello world”	Core
MOUSE_CONTROL	Move/click the mouse	“left click” · “move the mouse up”	Core

Media

Action	What it does	Example activation phrase(s)	Source
PLAY_MEDIA	Play a track on the named platform	“play Juicy by Notorious B.I.G. on Spotify” · “play lo-fi on YouTube”	Core
PAUSE_MEDIA	Pause playback	“pause” · “pause the music”	Core
STOP_MEDIA	Stop playback	“stop the music”	Core
NOW_PLAYING	Report the current track and where it’s playing (Spotify or headless-mpv YouTube audio)	“what’s playing” · “what song is this”	Core
NEXT_MEDIA	Next track	“next song” · “skip”	Core
PREVIOUS_MEDIA	Previous track	“previous track” · “go back a song”	Core
REPEAT_MEDIA	Repeat current track	“repeat this song”	Core
SHUFFLE_MEDIA	Toggle shuffle	“shuffle”	Core
MEDIA_CONTROL	Generic transport control (MPRIS)	“play/pause”	Core
SKIP_YOUTUBE_AD	Skip a YouTube ad when skippable	“skip the ad”	Core

Vision & screen

Action	What it does	Example activation phrase(s)	Source
SCREEN_READ_ANALYZE	Screenshot → vision model + OCR; answers questions about on-screen content	“what’s on my screen” · “can you see this” · “what season is on the screen”	Core
SCREEN_LOCATE	Find on-screen UI text/element	“find the submit button on screen”	Core
SCREENSHOT	Capture a screenshot	“take a screenshot”	Core
OCR_IMAGE	Extract text from an image file	“ocr photo.png” · “read the text in image.jpg”	Core
ANALYZE_IMAGE	Describe/analyse an image file (VLM)	“describe cat.jpg” · “analyse this image”	Core
ANALYZE_PDF	Read/summarise a PDF	“summarise report.pdf” · “analyse this pdf”	Core
ANALYZE_PDF_FOLDER	Analyse all PDFs in a folder	“analyse the pdfs in that directory”	Core
ANALYZE_CSV	Analyse a CSV/spreadsheet	“analyse data.csv”	Core
AMBIENT_VISION	Toggle continuous screen-watching	“watch my screen” · “stop watching”	Core
IMAGE_STATUS	Status of the latest image job	“what is the latest image status”	Core

Gaze (eye-tracking)

Action	What it does	Example activation phrase(s)	Source
GAZE_ENABLE	Enable webcam gaze control	“enable gaze control”	Core
GAZE_DISABLE	Disable gaze control	“disable gaze control”	Core
GAZE_CALIBRATE	Calibrate gaze tracking	“calibrate gaze”	Core
GAZE_STATUS	Show gaze tracker status	“gaze status”	Core
GAZE_CLICK	Click where the eyes rest	“open” / “left click” / “hit enter” (while gaze is on)	Core

Voice

Action	What it does	Example activation phrase(s)	Source
DICTATE	Start/stop dictation into the active field	“start dictation” · “stop dictating”	Core
TRANSCRIBE	Transcribe an audio file	“transcribe audio.wav”	Core
SPEAK	Speak text aloud (TTS)	“say good morning”	Plugin
LIST_VOICES	List installed voices and what’s downloadable (166 across 45 languages)	“what voices do you have” · “list voices” · “which accents can I get”	Core

Action	What it does	Example activation phrase(s)	Source
DOWNLOAD_VOICE	Download a voice/accent from the Piper library and switch to it	“download a British voice” · “get me a Scottish accent” · “install en_US-amy-medium”	Core
SET_VOICE	Switch the active TTS voice/accent (distinct from persona tone)	“use the alan voice” · “switch your voice to amy” · “use the robotic style voice” · “use the natural voice”	Core
CREATE_VOICE	Make a new ELI voice from a recording (wav/mp3/mp4 drop-in; XTTS-v2 clone)	“create a voice called Nova from clip.mp4” · “make an ELI voice from recording.wav”	Core
LISTEN_FOR_COMMAND	Start listening for a voice command	“listen” / the wake word “computer”	Core
DESIGN_VOICE	Build a custom style voice from a base voice with pitch, speed and effects	“design a voice called Nova, deep and slow” · “make a custom voice named Nova”	Core
WAKE_SET	Set your own wake word (any phrase); ELI persists it and trains the detector on it	“change the wake word to athena” · “set my wake word to atlas”	Core
WAKE_TRAIN	Train the local, self-supervised wake-word model (Piper-synth + noise/music augmentation; robust over background music)	“train the wake word”	Core
WAKE_ENROLL	Record your voice saying the wake word and retrain (personalisation, over music)	“enroll my wake word”	Core
TRAIN_VOICE	Learn your voice + tone — pitch/energy/rate, happy/angry/excited, and question-vs-statement; ELI then adapts its delivery to how you sound	“train my voice” · “learn how I speak”	Core
STT_DIAGNOSTICS	Mic/STT/TTS diagnostics (Whisper, wake word, ducking, Piper)	“run voice diagnostics” · “voice diagnostics” · “check the microphone” · “stt diagnostics”	Core

Files & notes

Action	What it does	Example activation phrase(s)	Source
CREATE_FILE	Create a file	“create a file notes.txt”	Core
CREATE_FOLDER	Create a folder	“make a folder called projects”	Core
READ_FILE	Read a file’s contents	“read notes.txt”	Core
LIST_DIR	List a directory	“list the files in ~/Documents”	Core
FILE_AUDIT	Audit files for issues	“audit my files”	Core
SUMMARIZE_FILE	Summarise a document/file	“summarise report.docx”	Core
CONVERT_DOCUMENT	Convert between document formats	“convert report.md to pdf”	Core
WRITE_NOTE	Write a quick note	“write a note saying buy milk”	Core
NEW_NOTE	Create a new note	“new note”	Plugin
LIST_NOTES	List saved notes	“list my notes”	Plugin
SEARCH_NOTES	Search notes	“search notes for budget”	Plugin
GET_CLIPBOARD	Read the clipboard	“what’s in my clipboard”	Core
SET_CLIPBOARD	Write to the clipboard	“copy this to the clipboard”	Core

Generation (docs/code)

Action	What it does	Example activation phrase(s)	Source
GENERATE_DOCUMENT	Grounded document via the multi-stage pipeline (evidence → plan/outline → sections → review→revise)	“generate a document about X” · “write a report on Y”	Core
CREATE_DOCUMENT	Multi-stage grounded document (plan→draft→review); single-pass fallback	“create a document on X”	Core
DATA_FABRICATOR	Generate synthetic/test data	“fabricate a test dataset for X”	Core
GENERATE_SCRIPT	Generate a runnable script via the coding agent	“write a bash script to monitor the GPU”	Core
GENERATE_PROJECT	Scaffold a multi-file project	“generate a project that does X”	Core
CODE_SOLVE	Solve a coding task (plan→DAG→verify→repair)	“solve this: …” · “implement function X”	Core
FIX_FILE	Find and fix bugs in a file	“fix the bugs in foo.py”	Core
EXAMINE_CODE	Tiered error scan of named files/codebase	“examine eli/memory/memory.py for errors” · “review the codebase for bugs”	Core

Action	What it does	Example activation phrase(s)	Source
CONFIRM_CODE_FIX	Apply the offered code fixes (pending-confirm)	“yes, fix it” · “fix the logic ones too”	Core
CANCEL_CODE_FIX	Decline the offered code fixes	“no, leave it”	Core
SHOW_DIFF	Show a code diff	“show the diff”	Core
OPEN_IN_IDE	Open a generated artifact in the IDE	(post-action; auto after generate)	Core
DOC_GENERATE	Alias of GENERATE_DOCUMENT	“generate a document about tides”	Core

System status & info

Action	What it does	Example activation phrase(s)	Source
CPU_USAGE	Current CPU usage	“cpu usage”	Plugin
RAM_USAGE	Current memory usage	“how much ram is free”	Plugin
GPU_STATUS	GPU/VRAM status	“gpu status”	Core
SYSTEM_STATS	Combined system stats	“system stats”	Plugin
HARDWARE_PROFILE	Detected hardware profile	“show hardware profile”	Core
GET_STATUS	General status	“status”	Core
TIME	Current time	“what time is it”	Core
GET_TIME	Current time (alt)	“tell me the time”	Core
DATE	Current date	“what’s the date”	Core
GET_DATE	Current date (alt)	“what day is it”	Core
GET_WEATHER	Weather for a place	“weather in Paris”	Plugin
NEWS_FETCH	Synthesised news digest; topic-scoped re-fetch after a briefing when you go deeper	“what’s the news” · “catch me up” · “go deeper into Hubble” · “more on the transformer story”	Core
WEB_SEARCH	Web search (Net-gated)	“search the web for X”	Plugin
WEB_FETCH	Read a linked web page and answer about it (Net-gated)	“https://github.com/ShadowLiC95/ELI_v2.0 what do you think?” · “summarise this https://example.com/article”	Core
MESSAGE_TIME_QUERY	Time-of-a-message query	“when did I say that”	Core
CHECK_CHRONAL_ALIGNMENT	Playful date/time sanity check	“check chronal alignment”	Core

Grounded introspection

Action	What it does	Example activation phrase(s)	Source
RUNTIME_STATUS	Live runtime: model, ctx, GPU layers, batch	“what are you running on” · “runtime status — everything”	Core

Action	What it does	Example activation phrase(s)	Source
RUNTIME_AUDIT	Full runtime audit (what's broken/missing)	"run a full runtime audit"	Core
IMPORT_AUDIT	Which imports are failing/missing	"what imports are failing"	Core
RESOLVE_RUNTIME_PATHS	Resolved paths of critical files	"show the resolved runtime paths"	Core
GUI_RUNTIME_AUDIT	Audit GUI wiring with file-read proof	"audit your gui file" · "scan the gui runtime wiring and prove every hook"	Core
DIAGNOSE_WRAPPERS	Show the executor middleware/wrapper chain	"show the executor wrapper chain"	Core
COGNITION_STATUS	Cognition runtime status	"cognition status"	Core
EXPLAIN_COGNITION_RUNTIME	Explain the cognition runtime verbatim	"explain your cognition runtime"	Core
EXPLAIN_GGUF_DIAGNOSTICS	Live GGUF load/inference compatibility report (runtime version, preflight, last load error)	"gguf errors" · "why won't nemotron load" · "empty response from the model"	Core
EXPLAIN_FAILURE_LOG	Grounded executor failure log from agent.sqlite3 (not GGUF console lines)	"what are the failures" · "show the failure log" · "what's tripping you up"	Core
EXPLAIN_LAST_FAILURE	Explain the most recent grounded or executor failure	"what was the last failure" · "why did that fail"	Core
MEMORY_STATUS	Memory subsystem status / counts	"how does your memory work"	Core
MEMORY_STATS	Memory statistics	"memory stats"	Core
EXPLAIN_MEMORY_RUNTIME	Explain the memory runtime verbatim (4 DBs)	"explain exactly how your memory works internally"	Core
AWARENESS_STATUS	What ELI is currently aware of	"what are you aware of"	Core
FRONTIER_STATUS	Full system wiring matrix	"run a full system wiring matrix"	Core
ELI_IDENTITY_AUDIT	Audit ELI's identity provenance	"audit your identity"	Core
CODE_CHANGES	Recent code changes	"what code has changed"	Core
NAME_SOURCE_AUDIT	Where ELI's knowledge of your name comes from	"how do you know my name"	Core
REASONING_MODE_STATUS	Current reasoning mode	"what reasoning mode are you in"	Core
EXPLAIN_ALL_REASONING_MODES	Describe all 5 reasoning modes	"explain all your reasoning modes"	Core

Action	What it does	Example activation phrase(s)	Source
SELF_REPORT	What ELI has recently done/worked on (code, commits, runtime)	“what updates have you made recently”	Core

Memory & profile

Action	What it does	Example activation phrase(s)	Source
MEMORY_STORE	Store a durable fact	“remember that my sister’s name is Anna”	Core
MEMORY_RECALL	Recall stored facts	“what do you remember about my car”	Core
MEMORY_FORGET	Delete matching memories and everything derived from them, after confirmation	“forget that my locker is 212”	Core
PERSONAL_MEMORY_SUMMARY	Summary of what ELI knows about you	“what do you know about me”	Core
PERSONAL_MEMORY_DEEP_EXPLAIN	Deep, sourced profile explanation	“explain everything you know about me and where it’s stored”	Core
USER_IDENTITY_SUMMARY	Who the user is	“who am I”	Core
REFRESH_USER_INFO	Re-extract the user profile	“refresh my user info”	Core
SET_USER_NAME	Set/correct the user’s name	“my name is Alex” · “call me Alex”	Core
USER_INFO_REPORT	Full report of the living user profile; reached through the intent resolver	(no dedicated phrase)	Core

Self-maintenance

Action	What it does	Example activation phrase(s)	Source
SELF_ANALYZE	Analyse own failures/metrics	“analyse your failures”	Core
SELF_IMPROVE	Run a self-improvement patch cycle	“improve yourself”	Core
SELF_PATCH	Apply a self-improvement patch	“patch yourself”	Core
SELF_REPAIR_PLAYBOOK	Grounded repair decision guide (self help)	“self help” · “repair playbook” · “maintenance playbook”	Core

Action	What it does	Example activation phrase(s)	Source
AUTOPILOT_DEBUG	Turn a failure into a plan: root cause, affected files from git history, rollback, patch and validation	“debug this error: ValueError x in eli/core/paths.py” · “autopilot debug this traceback”	Core
CODEBASE_GRAPH	Import graph of ELI’s own source: what depends on what	“show the codebase graph”	Core
TIMESTAMP_DIAG	Run real clock and timestamp diagnostics instead of guessing	“check the timestamps” · “investigate the timestamp problem”	Core
SELF_IMPROVEMENT_LOG	Show the self-improvement log	“show your self-improvement log”	Core
SELF_UPGRADE	git pull → deps → rebuild indexes	“upgrade yourself”	Core
SELF_UPDATE	Self-update (control-contract)	“update yourself”	Core
SELF_TEST	Run internal self-tests	“run a self test”	Core
RUN_TESTS	Run the pytest suite and summarise the results document	“run the test suite” · “generate a test report”	Core
GENERATE_TESTS	ELI writes + sandbox-verifies behavioural tests for its own functions (Phase 4)	“generate tests for your code” · “grow your test coverage”	Core
LORA_STATUS	Report LoRA fine-tune readiness (preflight: modules, base model, reviewed data)	“lora status” · “is lora ready”	Core
LORA_TRAIN	Run the LoRA training pipeline DAG (pre-flight→build→train→eval); dry-run from chat, real training via the overnight task	“train a lora” · “fine-tune yourself”	Core
ORCHESTRATION_STATUS	Explain the agent DAG orchestrator: execution layers, dependencies, critical path, and last run	“orchestration status” · “show your agent dag”	Core
TEST_REVIEW	Run the suite, back up the report + write an errors file, summarise results, and offer result-driven fix options	“test review” · “run the tests and tell me what to fix”	Core

Plugins

Action	What it does	Example activation phrase(s)	Source
PLUGIN_STATUS	Status of one plugin; reached through the intent resolver (“list plugins” lists them all)	(no dedicated phrase)	Core
MCP_LIST	List configured MCP servers	“list mcp servers”	Core
MCP_STATUS	Status of MCP servers and their tools	“mcp status”	Core
MCP_TOOLS	List the tools MCP servers expose	“list mcp tools”	Core
MCP_CALL	Call a tool on an MCP server (the server runs outside ELI’s network guard)	“call the mcp tool read_file”	Core
MCP_ADD	Add an MCP server (installed switched off until you enable it)	“add an mcp server”	Core
MCP_REMOVE	Remove an MCP server	“remove the mcp server github”	Core
MCP_DOCTOR	Check MCP servers: launch preflight and handshake	“mcp doctor” · “check my mcp servers”	Core
PLUGIN_LIST	List installed plugins	“list plugins”	Core
PLUGIN_SEARCH	Search for plugins	“search plugins for weather”	Core
PLUGIN_INSTALL	Install a plugin	“install the X plugin”	Core
PLUGIN_UNINSTALL	Uninstall a plugin	“uninstall the X plugin”	Core
PLUGIN_ENABLE	Enable a plugin	“enable the X plugin”	Core
PLUGIN_DISABLE	Disable a plugin	“disable the X plugin”	Core

Tasks, time & planning

Action	What it does	Example activation phrase(s)	Source
SCHEDULE_TASK	Schedule overnight/timed work (code/research/etc.)	“research X overnight” · “build Y at 2am”	Core
SET_ALARM	Set an alarm	“set an alarm for 7am”	Core
SET_TIMER	Set a timer	“set a timer for 10 minutes”	Core
ADD_EVENT	Add a calendar event	“add an event tomorrow at 3pm”	Core
LIST_EVENTS	List events	“list my events”	Core
REMOVE_EVENT	Remove a calendar event or reminder	“cancel the dentist appointment”	Core
POMODORO_START	Start a pomodoro	“start a pomodoro”	Plugin
POMODORO_STOP	Stop the pomodoro	“stop the pomodoro”	Plugin
POMODORO_STATUS	Pomodoro status	“pomodoro status”	Plugin
BACKGROUND_JOBS	List background/scheduled jobs	“show background jobs”	Core

Action	What it does	Example activation phrase(s)	Source
CHECK_JOB	Check a specific job	“check job 5”	Core
HABIT_STATUS	Show learned habits	“show my habits”	Core
CONFIRM_HABIT	Approve a proposed habit	“yes” (to a habit offer)	Core
DECLINE_HABIT	Decline a proposed habit	“no” (to a habit offer)	Core
EXECUTE_GOAL	Execute a stored goal	“execute goal X”	Core
RESUME_TASK	Recover a saved task record and state its real staleness	“continue the RAIMS project” · “resume the migration task”	Core

Proactive

Action	What it does	Example activation phrase(s)	Source
PROACTIVE_START	Start the proactive daemon	“start proactive mode”	Core
PROACTIVE_STOP	Stop the proactive daemon	“stop proactive mode”	Core
PROACTIVE_STATUS	Proactive daemon status	“proactive status”	Core
GET_PROPOSALS	ELI’s self-generated proposals/goals from observed signals	“do you have any proposals for me” · “what’s on your agenda”	Core
MORNING_REPORT	Daily briefing (news/weather/agenda)	“morning report” · “give me my briefing”	Core

Shell & advanced

Action	What it does	Example activation phrase(s)	Source
RUN_CMD	Run a shell command (gated/confirmed)	“run the command: ls -la”	Core
SHELL_EXEC	Execute shell (gated/confirmed)	“execute: df -h”	Core
SEQUENCE	Run a multi-step action sequence	“do X then Y then Z”	Core
ROUTING_FAULT_EXPLAIN	Explain why an input failed to route	“why did that fail to route”	Core
NOOP	Internal no-op (ignored fragment guard)	(internal — fragmentary input is dropped)	Core

Aliases & internal actions (not directly user-triggered)

Synonyms normalised to a routable action, post-actions, or internal pipeline steps:

ANSWER, CHECK_TARGET_STATUS, CLOSE_APPLICATION, CREATE_DOC, CREATE_SCRIPT, DIRECT_RESPONSE, EXIT_APP, GENERATE_CODE, LAUNCH_APP, OPEN_APPLICATION, PREPARE_REMEDIATION, QUIT_APP, SAY, VOICE_DIAGNOSTICS, WRITE_CODE, WRITE_DOCUMENT, WRITE_SCRIPT

Coverage

- Routable actions documented: **214/214**